Developing a Self-Driving RC Car

SURF 2025 Report

Rodi Woldeyesus and Sam Henry, Ph.D.

1. Intro, Motivation and Significance

This project focuses on developing a self-driving RC car. The goal is to make learning about autonomous driving technology and machine-learning engaging for students. This hands-on approach allows students to gain practical experience with problem-solving, programming, robotics, and machine learning concepts. Given that the self-driving car market was valued at around $54.23 billion in 2019 and is projected to reach $555.67 billion by 2026 (Garsten), there's a clear need for education in this area. Many students, however, still lack opportunities for hands-on learning in autonomous vehicles. My project aims to bridge this gap by providing practical experiences in machine vision (UPCEA).

In this document, you'll find a detailed overview of the tools and software used, what we've learned throughout the project, and the next steps for future work. This should serve as a comprehensive guide for anyone interested in continuing this project in the future. I provide a guide, and code available here: https://github.com/ml-rmc/donkey_driving)

2. Hardware and Software Selection, Configuration, and Calibration

2.1 Development Platforms

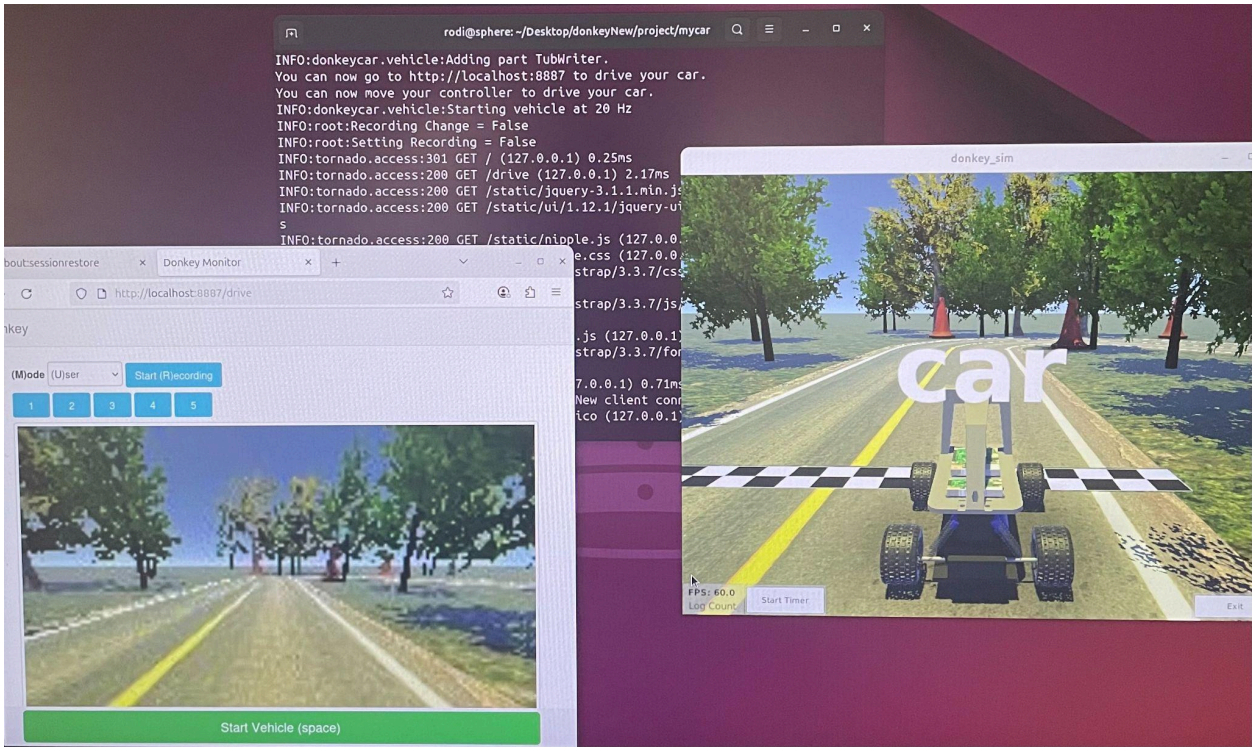
Developing an autonomous vehicle is complex, and while you could code everything from scratch, there are several platforms available that provide features such as hardware interfaces, simulation environments, data management, classes for training and running models, and guides and community support. Three popular platforms include Amazon Web Services (AWS) DeepRacer, Robot Operating System (ROS), and Donkey Car.

AWS DeepRacer (https://aws.amazon.com/deepracer/) is designed to teach users about machine learning through racing simulations. It attempts to make getting started simple by providing two fully assembled and tested hardware implementations that work natively with a cloud-based software environment (AWS DeepRacer). The software environment provides a 3D simulation and allows you to train your models with a focus on reinforcement learning methods. There are tutorials, however as of early 2025, Amazon is phasing out official support for the platform and instead hopes community-driven efforts will keep the platform alive. Furthermore, the platform is very restrictive and pushes users to use AWS resources. These restrictions make getting started simple because there aren't many options, and all the software works with the hardware. However it ties users to the official platforms and software. Since official support is ending, it makes AWS deepracer's future uncertain. The source code was made publicly available (https://github.com/aws-deepracer/aws-deepracer), but these concerns remain.

Robot Operating System (ROS) (https://www.ros.org/) is the long-standing de facto standard for developing robotic applications. It is an open-source comprehensive set of software libraries for integrating hardware, simulation, data management, and training. It has a very active community and extensive guides, including many textbooks. ROS is a generalized robotics development environment meaning it can be used to develop robotic applications ranging from robotic arms to self-driving cars. This flexibility makes ROS a powerful tool, but also means that it has a steep learning curve, and the lack of focused resources for self-driving can make it challenging for someone new to the field to get started.

Donkey Car (https://www.diyrobocars.com/) is an open-source software suite focused on developing self-driving RC cars. It offers software for various hardware interfaces, a 3D simulation environment, and code for data collection, data management, and training deep-learning and creating more traditional machine vision-based models. It also has extensive guides and tutorials, an active community, including Discord channel (https://discord.gg/p8Khj5cQ) for trouble-shooting, and several meet-ups and race events throughout the country. Donkey Car's focus on self-driving makes it approachable for beginners, and there are several pre-made hardware options available. It also offers flexibility in working with custom-built cars. This focus, flexibility, and community support make Donkey Car an excellent choice for beginners.

2.2 Donkey Gym Simulator



The Donkey Car Simulator, also called Donkey Gym or Donkey Simulator is a crucial component of the Donkeycar project, designed to create a virtual environment where users can

simulate the behavior of self-driving cars. This simulator is built on the OpenAI Gym, which is a toolkit that provides a standard way to develop and test machine learning algorithms. The main purpose of the simulator is to allow users to experiment with and refine their models in a safe, controlled setting, without the risks and costs of physical testing.

I initially tried to set up the Donkeycar app and simulator on a Windows PC. Although I was able to launch the simulator, I faced difficulties when trying to configure the joystick controls. The system automatically attempted to direct the joystick input to the /dev/input directory, which is specific to Unix-like operating systems such as Linux. In Windows, input devices like joysticks are managed through the Windows Device Manager, which recognizes and lists connected devices but does not expose them through a directory structure like /dev/input. This means that, although my joystick was recognized in the Device Manager, I could not locate a corresponding directory path for it. This experience highlighted the importance of using a compatible operating system, which led me to switch to Linux.

Once I moved to Linux, I began the installation process of Donkeycar. The first step was to make sure that the correct version of Python was installed. The software requires Python versions greater than or equal to 3.11.0 but strictly less than 3.12. This means that Python 3.11.0 is the minimum version supported, and any version up to but not including 3.12 is compatible with Donkeycar. Next, I created a virtual environment using pip, a tool for installing and managing software packages. A virtual environment is like a separate workspace that keeps project dependencies organized and prevents conflicts with other projects.

After setting up the environment, I cloned the Donkeycar repository from GitHub. I downloaded the simulator files for Linux and organized them into a specific folder. The Donkey Gym acts as a bridge between the Donkeycar application and the virtual environment, allowing them to communicate effectively.

To integrate the Donkey Gym, I cloned the gym-donkeycar repository. This repository contains additional code that helps connect the simulator with the Donkeycar application, making it easier to test and train the models.

Creating a new car configuration was straightforward. A car configuration defines how the virtual car behaves, including its speed, steering, and other parameters. I edited a file called myconfig.py to set this up, specifying the path to the simulator and defining the environment for the simulated track.

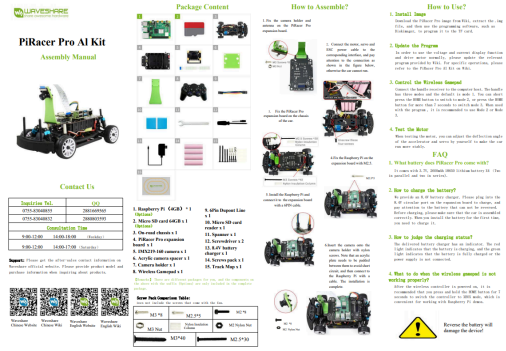
With everything set up, I was ready to test the simulator. I launched it using a specific command, which also connected me to a web interface. This interface allows you to control the virtual car easily and see how it performs. The simulator supports joystick integration, making the driving experience more intuitive and similar to playing a video game.

As I drove the virtual car, it recorded data in a specific folder. This data collection is essential for training the model to imitate my driving behavior. After collecting enough data, I started the training process, enabling the model to learn from my inputs. Once training was complete, I tested the model to see how well it performed in the simulated environment.

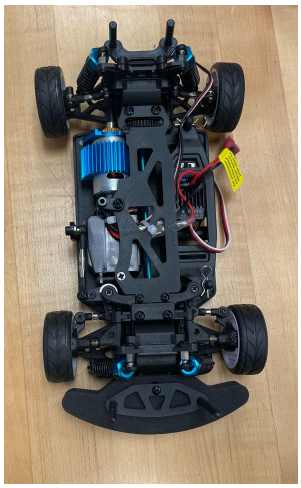
2.3 The RC Car

When choosing a car for your Donkeycar project, there are many options. Donkeycar supports various models for autonomous driving. A good starting point is the official website, which lists compatible vehicles and setup guidelines.

While some products claim compatibility with the Donkeycar software, it is essential to verify their functionality to avoid potential issues. I initially purchased the Xiao R race car, but faced challenges with the I2C board recognition. I then switched to the Waveshare Pi Racer Pro (https://www.waveshare.com/piracer-pro-ai-kit.htm), recommended on the DonkeyCar website (Donkey Car).



I followed the Waveshare assembly instructions to put the car together, but when it came to downloading the software, I found the version on the Waveshare site was outdated. I had to redownload the software from the actual Donkeycar site (https://github.com/autorope/donkeycar), which should be the go-to source for the latest version.



2.4 The Single Board Computer

I used a Raspberry Pi 4B as the onboard computer, which is well-supported in the DonkeyCar community. Alternatives are NVIDIA Jetson, Orin, and other similar models. While the PiRacer Pro supports an NVIDIA Jetson RC Car, official support for the NVIDIA Jetson is ending, and conflicting driver issues and other other software support problems have been reported.

I used the 64-bit Raspberry Pi OS Bookworm for the latest version of Donkeycar(5.1). The installation began with the Raspberry Pi imager, where I was able to configure my Wi-Fi settings and hostname.

After booting the Raspberry Pi, I accessed it via SSH to perform updates and upgrades, ensuring all software components were current. I also used the raspi-config utility to enable I2C and expand the filesystem.

Finally, I created a virtual environment to manage the dependencies and installed Donkeycar using pip. With the hardware assembled and the software in place, I tested the car's components to ensure everything was working properly. I verified that the camera was functional and confirmed that TensorFlow was installed correctly.

2.5 RC Car Calibration

Calibrating your car is essential for ensuring smooth and consistent driving. This process involves setting key parameters, particularly the PWM (Pulse Width Modulation) values for steering and throttle. If your car has a steering servo, Donkeycar needs the PWM values for full left and right turns. Similarly, if you're using an ESC (Electronic Speed Controller), you'll need the PWM values for full forward throttle, stopping, and full reverse (Donkey Car).

If calibration is skipped, the car may still operate, but it could lead to erratic behavior, such as poor steering response or inconsistent acceleration, ultimately affecting performance and safety.

The calibration process includes two main steps: regular calibration and fine-tuning. For steering calibration, you'll identify the PWM values that allow for full left and right turns without causing the servo to whine. This typically involves a trial-and-error approach, where you input different PWM values until you find the ones that achieve the desired movement.

For throttle calibration, you'll establish PWM settings for forward motion, stopping, and reverse, as ESCs require specific signals for proper operation. Adjusting these settings ensures that the car can accelerate and reverse smoothly (Donkey Car).

After initial calibration, fine-tuning is necessary. First, ensure the car drives straight when no steering input is applied. If it veers left or right, adjust the corresponding PWM values in your configuration file to bring it closer to neutral.

Next, measure the turning radius for both left and right turns to achieve balance. This may involve measuring how far the car turns at different PWM settings and adjusting accordingly (Donkey Car).

3. Self-Driving Models

In my Donkeycar project, I use two main methods to control the car: Traditional Machine Vision and Deep Learning. Machine vision uses image processing methods, line detection, which allows the car to follow paths without needing a lot of training data. Whereas Deep learning is based purely on data and uses a convolutional neural network (CNN) to help the car navigate by mapping an image to throttle and steering values.

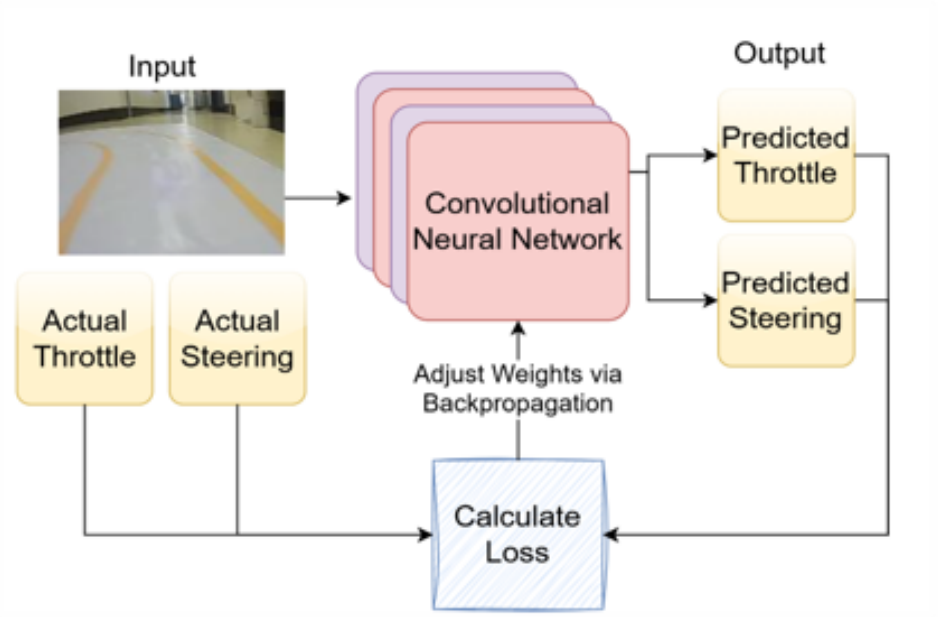
3.1 Machine Vision

Machine vision enables computers to interpret visual information. In my Donkeycar project, I used machine vision for line detection, allowing the car to navigate autonomously along paths. Unlike the deep learning autopilot, which requires extensive training data from human driving, the machine vision autopilot uses traditional computer vision techniques. These include color space conversion and edge detection to identify key features in camera images. The machine vision approach allows for immediate implementation without prior data collection. I can customize the algorithm to suit the track conditions, making adjustments as needed.

The car will capture images in real-time, processing them to detect lines and translate this information into steering and throttle values, operating 20 times per second. This responsiveness is practically advantageous in changing outdoor environments.

3.2 Deep Learning

Deep Learning is a subset of artificial intelligence that teaches computers to recognize patterns in data. In my Donkeycar project, I used deep learning to enable the car to navigate autonomously by making decisions based on visual input. This is a form of imitation learning, where the car learns to drive based on user-provided input (Bojarski). It maps images from the camera to throttle and steering values which control the car.



This image illustrates the architecture of the convolutional neural network (CNN) used in my project. The input layer receives image data, which is processed through a convolutional neural network which outputs the predicted throttle and steering values.

In a computer, images are represented as 3D matrices of values, where each value corresponds to the intensity of red, green, and blue (RGB) pixels. When the car captures an image, this data is fed into the CNN. The CNN analyzes the image and predicts the throttle and steering values needed for navigation.

The goal of deep learning is to minimize the error between predicted and actual values, specifically the throttle and steering commands needed for the car to function correctly. This is achieved through a process called training, where the model learns how to transform input images into the appropriate output values. The transformation is a mathematical function that maps image data to the required commands.

Therefore, this training procedure requires data to learn. This data is collected through the car's camera while it is driven manually or autonomously. The result is a dataset in which each image has a corresponding throttle and steering value associated with it. The training algorithm will minimize loss over this dataset. Generating a high-quality and varied training dataset is therefore critical to creating a successful deep learning model. The model will learn to mimic the training data, and will therefore at best perform on par with that data provided.

The model’s performance is evaluated using a loss function, which quantifies the difference between predicted outputs and actual outputs. A common loss function used in this context is the sum of squared errors, which calculates the total of the squared differences between the predicted and actual throttle and steering values.

To improve the model, we utilize a technique called backpropagation. During this process, the model calculates the error and adjusts its internal parameters, known as weights, to reduce future prediction errors. Essentially, the model learns from its mistakes, gradually refining its predictions over time.

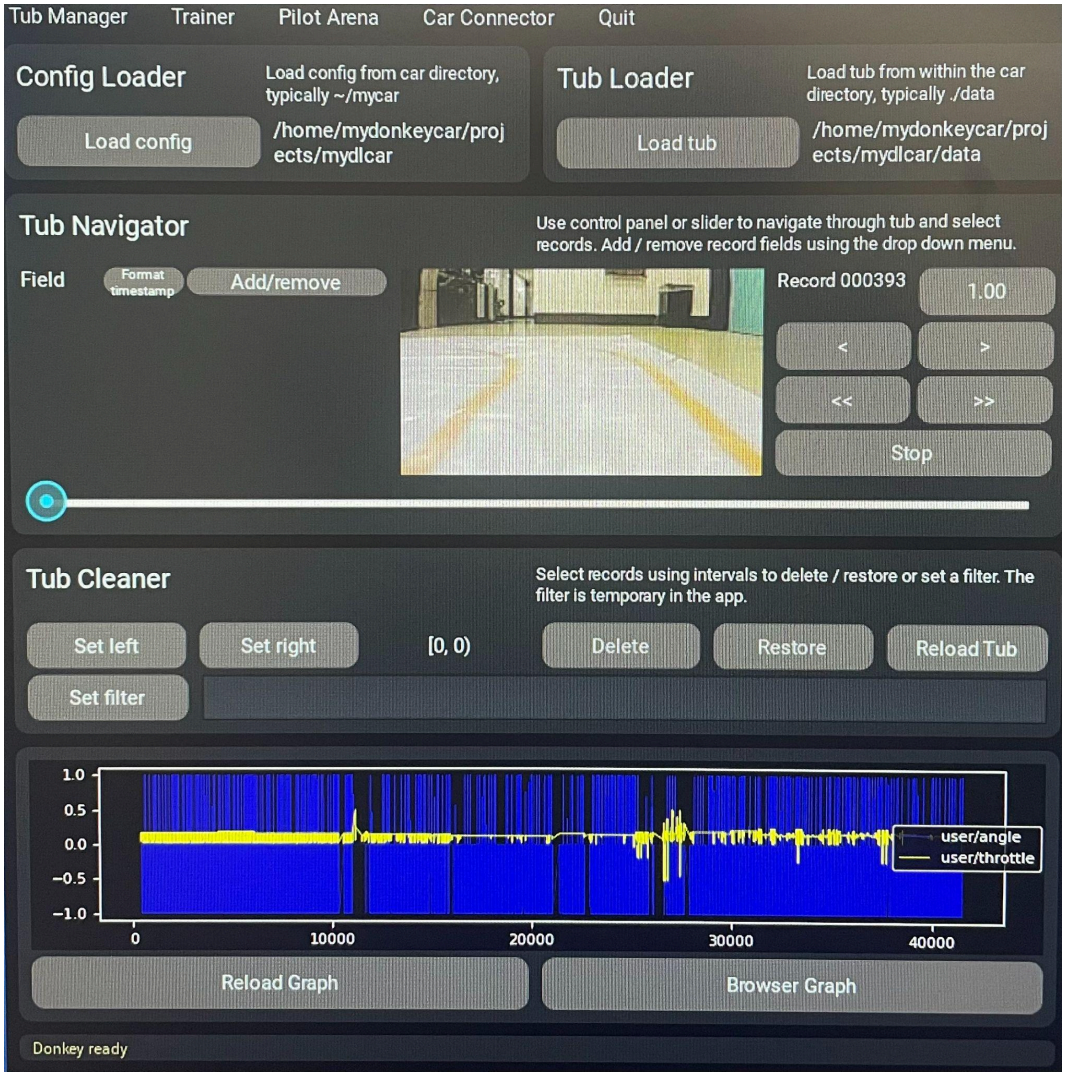
In summary, the input to the system is the image data represented as a 3D matrix, while the output consists of the predicted throttle and steering values. The deep learning model learns the transformation from input to output through training and backpropagation, ultimately enabling the car to navigate autonomously.

3.3 Data Collection and Editing

In my Donkeycar project, data collection and editing are crucial for model performance. High-quality data is essential since the algorithm learns to mimic the training data.

Data collection involves gathering images and corresponding control commands (throttle and steering values) while driving. This data is stored in "tubs," structured datasets that facilitate training. Each tub contains image data paired with actions, allowing the model to learn the relationship between visual input and driving behavior.

I use the command donkey train --tub ./data to start training on the collected data, ensuring the model has access to the necessary information (Donkey Car).



Data editing refines the dataset to improve learning capabilities. This includes filtering out irrelevant or low-quality data, which might negatively affect training. The Donkey UI offers tools for easy record manipulation, allowing users to filter data, delete poor-quality records, and visualize dataset quality.

High-quality training data is vital. A well-curated dataset helps minimize prediction errors. The model's performance is evaluated using a loss function, like the sum of squared errors, which measures the difference between predicted and actual outputs. Ensuring a representative dataset allows the model to generalize better to new driving scenarios

4. Results, Conclusions, and Future Work

During my Donkeycar project, I faced several challenges with the camera. Initially, the camera was not working properly. To resolve this, we swapped components and used the Xiao R Race Car Geek's webcam with the Waveshare setup, which improved performance.

I also encountered issues with the Donkey UI not loading. I found that removing the previously trained models fixed this problem. This allowed the interface to work correctly and enabled us to load the data tubs.

The Discord community on the Donkeycar website was incredibly helpful. Their support and insights provided valuable guidance throughout the project.

OpenAI Gym focuses on developing reinforcement learning methods, so therefore, we could use the Donkey Gym for reinforcement learning. This would be a good future direction. We also plan to generate more, better data, and develop more advanced models that combine the machine-vision and deep learning models.

Works Cited

“AI Algorithms for Autonomous Vehicles: How Advanced Data Solutions Improve Vehicle Perception and Decision-Making.” Voxelmaps, 1 Aug. 2024, www.voxelmaps.com/news/ai-algorithms-for-autonomous-vehicles. Accessed 3 Mar. 2025.

“Autonomous Driving: What is ADAS Sensor Fusion?” Car ADAS Solutions, 29 Nov. 2023, www.caradas.com/adas-sensor-fusion/. Accessed 3 Mar. 2025.

Donkey Car. Donkeycar Documentation, https://docs.donkeycar.com/. Accessed 3 Mar. 2025.

Garsten, Ed. “Sharp Growth in Autonomous Car Market Value Predicted but May Be Stalled by Rise in Consumer Fear.” Forbes, 13 Aug. 2018, www.forbes.com/sites/edgarsten/2018/08/13/sharp-growth-in-autonomous-car-market-value-predicted-but-may-be-stalled-by-rise-in-consumer-fear/.

Jesus, Ayn de. “AI for Self-Driving Car Safety – Current Applications.” EMERJ, 16 Jan. 2019.

“Racing Simulator Software - AWS DeepRacer.” AWS, https://aws.amazon.com/deepracer/. Accessed 3 Mar. 2025.

“The Effect of Autonomous Vehicles on Education.” UPCEA, https://upcea.edu/wp-content/uploads/2017/08/The-Effect-of-Autonomous-Vehicles-on-Education.whitepaper.december16.pdf. Accessed 3 Mar. 2025.

Bojarski, Mariusz, et al. "End to end learning for self-driving cars." *arXiv preprint arXiv:1604.07316* (2016).